SIMATS ENGINEERING
Engineer to Excel

SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO1 – Assessment Tool 1: IT Application-Based Problem Solving – Scalability Audit of an E-Commerce Platform
Course Outcome Assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including Master Theorem and substitution method.
Duration	3 Days
Mode	Individual
Student Reg. No.	1971260034
Student Name	Chundi.Bhavagna

Instructions: Answer all parts. Show the basic operation, every step of each derivation and the final asymptotic result, and interpret what each result means for the platform. Assume the platform's servers execute 10^8 basic operations per second.

IT APPLICATION-BASED PROBLEM

Scenario – Scalability Audit of "ShopSphere", an Online Shopping Platform: ShopSphere is a growing e-commerce platform whose product catalogue currently holds n = 100,000 products and is expected to reach n = 1,000,000 within a year. Before the expansion, the CTO has asked you, as the platform's algorithm engineer, to audit five back-end modules, predict how each will behave as n grows, and recommend which modules must be redesigned. The modules are described below.

Module M1 – Duplicate-Listing Detector: Sellers sometimes upload the same product twice. Every night, the following routine compares every product with every later product.

```
FOR i ← 0 TO n − 2 DO
   FOR j ← i + 1 TO n − 1 DO
      IF Product[i].title = Product[j].title AND Product[i].seller =
Product[j].seller THEN
         MarkDuplicate(j)
      END IF
   END FOR
END FOR
```

Module M2 – API Rate Limiter: The public API audits request windows of doubling size using the routine below. A team member claims that it is Θ(n log n) because the outer loop runs about log n times and the inner loop runs up to n times.

```
i ← 1
WHILE i < n DO
   FOR j ← 0 TO i − 1 DO
      Audit(Request[j])          // basic operation
   END FOR
   i ← i × 2
END WHILE
```

Module M3 – Product Search: Two versions exist. Version S1 scans the unsorted catalogue sequentially until the product is found. Version S2 searches a sorted product index by comparing the key with the middle entry and discarding half the index after each comparison; its comparison count satisfies $T(n) = T(n/2) + 1$, $T(1) = 1$.

Module M4 – Recommendation Engine: The engine splits the customer–product data into four parts of size n/2 (because of replicated partitions) and merges the partial results in linear time: $T(n) = 4T(n/2) + n$. A proposed redesign removes the replication, giving $T(n) = 2T(n/2) + n$.

Module M5 – Sales Analytics Engine: The analytics engine splits the sales data into halves recursively and sorts the records by revenue at every level, giving $T(n) = 2T(n/2) + n \log n$. Separately, profiling shows that the nightly report generator performs $f(n) = 3n^2 + 10n \log_2 n + 50$ operations.

(a) Understand the Problem: For each of the five modules, identify the input size and the basic operation that dominates the running time, explain where it occurs, and state how you expect its execution count to grow as n increases. Justify each choice briefly.

(b) Analyse the Iterative Modules: For M1, derive the exact number of comparisons as a function of n and state the time and space complexity. For M2, derive the exact number of Audit operations, decide whether the team member's $\Theta(n \log n)$ claim is correct and give the tight bound. Verify your M2 result for n = 1,024.

(c) Solve a Recurrence by Substitution: Solve the recurrence for search version S2 using backward substitution, showing every intermediate step, and derive the exact closed form. State the time complexity and the space complexity of both an iterative and a recursive implementation.

(d) Apply the Master Theorem: For M4, apply the Master Theorem to both the current design and the proposed redesign, clearly identifying a, b, f(n), n^(log_b a) and the case used, and explain what happens to the running time when the data doubles. For M5, determine whether any of the three standard cases applies to $T(n) = 2T(n/2) + n \log n$; if not, derive the complexity using the extended Master Theorem or a recursion tree.

(e) Analyse Best, Average and Worst Cases: For search versions S1 and S2, identify the input condition that produces the best, average and worst case and give the number of comparisons in each. For S1, derive the average number of comparisons when the probability that the searched product exists is p, and evaluate it for n = 1,000,000 and p = 0.5.

(f) Prove an Asymptotic Bound: Using the formal definitions, prove that the report generator's cost $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$ by finding constants c_1, c_2 and n_0, and show that f(n) is not O(n).

(g)

Scalability Decision: Estimate the number of basic operations and the running time of M1, M2, S2, M4 (both designs) and M5 for n = 1,000,000. Identify the modules that will fail to meet a nightly processing window of 10 minutes, and recommend, with justification, which modules must be redesigned and how.

Rubrics – Application-Based Question

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach

THEORY – PAGE 1: INTRODUCTION TO ALGORITHM ANALYSIS

Meaning of an algorithm

An algorithm is a finite sequence of clear steps used to solve a problem or complete a task. In computer science, algorithms are used for searching, sorting, data processing, recommendation systems, and many other applications.

Algorithm analysis studies how the time and memory requirements of an algorithm change when the input size increases. It helps developers choose a suitable method before a system is used with large amounts of data.

Input size and basic operation

Input size is commonly represented by n. For an online shopping platform, n may represent the number of products, customer records, or sales entries being processed.

A basic operation is the main operation whose repeated execution determines the running time. Examples include comparing two product titles, checking a request, comparing a search key, or processing one record.

Why analysis matters

An algorithm that works well for a small catalogue may become too slow when the catalogue grows. Analysis helps predict this change and identify modules that need redesign before they affect the whole platform.

TIME AND SPACE COMPLEXITY

Time complexity

Time complexity describes how the number of basic operations grows with input size. It is usually expressed using asymptotic notation rather than the exact time on one particular computer.

For example, a linear algorithm performs a number of operations proportional to n. If the input doubles, its work is expected to roughly double. A quadratic algorithm performs work proportional to n^2, so doubling the input can make its work about four times larger.

Space complexity

Space complexity describes the extra memory required as input size grows. It can include temporary arrays, recursion stacks, stored results, and other additional data structures.

An iterative algorithm may use constant extra space if it only stores a few variables. A recursive algorithm often uses stack space for its active function calls. For a recursive binary search, the call stack grows logarithmically with n

Practical interpretation

Time and space must both be considered. An algorithm may be fast but use too much memory, or use little memory but take too long. A good design balances these costs according to the platform's requirements.

ASYMPTOTIC NOTATIONS

Big-O notation: O(g(n))

Big-O gives an asymptotic upper bound. It describes a function that does not grow faster than a constant multiple of g(n), for sufficiently large n. It is commonly used to describe an algorithm's worst-case growth rate.

Big-Omega notation: □(g(n))

Big-Omega gives an asymptotic lower bound. It states that the function grows at least as quickly as a positive constant multiple of g(n), for sufficiently large n.

Big-Theta notation: □(g(n))

Big-Theta gives a tight asymptotic bound. A function f(n) is □(g(n)) when it is both O(g(n)) and □(g(n)). This means the two functions have the same growth order, ignoring constant factors and lower-order terms.

Common growth rates

Common orders from slower to faster growth are □(1), □(log n), □(n), □(n log n), □(n^2), □(n^3), and □(2■). For large inputs, the difference between these growth rates can be substantial.

ANALYSING LOOPS AND SEARCHING

Single and nested loops

A loop that performs one constant-time operation for each of n items usually has □(n) time complexity. If one loop runs n times inside another loop that also runs n times, the total number of iterations is approximately n^2, giving □(n^2) complexity.

For a loop whose counter doubles each time, the number of iterations is logarithmic because the values progress as 1, 2, 4, 8, and so on. A nested loop that performs i operations while i doubles performs 1 + 2 + 4 + … + 2■ operations, which is a geometric sum.

Sequential search

Sequential search checks items one by one until the target is found or the list ends. In the best case, the target is the first item, requiring one comparison. In the worst case, it is the last item or is absent, requiring up to n comparisons.

For a successful search where each position is equally likely, the average number of comparisons is (n + 1) / 2. If the target may be absent, the average depends on the probability of a successful search and on the assumed distribution of target positions.

Binary search

Binary search requires sorted data. It compares the target with the middle item and discards half of the remaining search interval after each comparison. The number of comparisons grows logarithmically, giving □(log n) time.

RECURRENCES AND SUBSTITUTION

What is a recurrence?

A recurrence expresses the cost of a problem in terms of smaller instances of the same problem. It is useful for recursive algorithms such as binary search, merge sort, and divide-and-conquer methods.

Example: binary search recurrence

For binary search, each step reduces the input size from n to n/2 and performs one comparison. The recurrence is T(n) = T(n/2) + 1, with a constant base case such as T(1) = 1.

Using backward substitution gives T(n) = T(n/2) + 1; then T(n/4) + 2; then T(n/8) + 3. After k steps, T(n) = T(n/2■) + k.

Finding the stopping point

The recursion stops when n/2■ = 1. Rearranging gives 2■ = n, so k = log■ n. Therefore the number of comparisons grows in proportion to log■ n, and the time complexity is □(log n).

An iterative version avoids recursive call-stack space and can use □(1) auxiliary space. A recursive version uses □(log n) stack space because the depth of the recursion is logarithmic.

MASTER THEOREM

General form

The Master Theorem helps analyse divide-and-conquer recurrences of the form T(n) = aT(n/b) + f(n). Here, a is the number of recursive subproblems, n/b is the size of each subproblem, and f(n) is the non-recursive work done at each level.

Compare the terms

The key comparison is between f(n) and n^(log_b a). The term n^(log_b a) represents the combined growth associated with the recursive subproblems. The standard cases compare f(n) with this term.

Typical cases

Case 1: If f(n) is polynomially smaller than n^(log_b a), the recurrence is □(n^(log_b a)).

Case 2: If f(n) has the same order as n^(log_b a), allowing a logarithmic factor, the result includes that logarithmic factor.

Case 3: If f(n) is polynomially larger and the regularity condition holds, the recurrence is □(f(n)).

Applying the theorem carefully

For T(n) = 4T(n/2) + n, a = 4 and b = 2, so n^(log■ 4) = n². The non-recursive term n is smaller than n², so the result is □(n²). For T(n) = 2T(n/2) + n, the two comparison terms are both n, giving □(n log n).

: RECURSION TREES AND COMPLEXITY PROOFS

Recursion tree method

A recursion tree represents each recursive call as a node. The work performed at each level is added, and the level totals are summed until the base cases are reached. This method is helpful when a recurrence does not fit a standard Master Theorem case directly

.

Example with an extra logarithm

Consider T(n) = 2T(n/2) + n log n. At a level where subproblem sizes are approximately n/2■, there are 2■ subproblems. The total non-recursive work at that level is proportional to n(log n □ j).

There are approximately log■ n levels. Adding the level costs produces a sum proportional to n(log n + (log n □ 1) + … + 1), which is □(n(log n)²).

Proving a tight bound

To prove f(n) = □(g(n)), choose positive constants c■, c■, and n■ such that 0 □ c■g(n) □ f(n) □ c■g(n) for all n □ n■.

For f(n) = 3n² + 10n log■ n + 50, the quadratic term dominates as n grows. Since n log■ n and the constant term grow more slowly than n², the function has tight bound □(n²). It is not O(n), because its quadratic growth eventually exceeds every fixed constant multiple of n.

: SCALABILITY AUDIT AND DESIGN DECISIONS

Meaning of scalability

Scalability is the ability of a system to handle increasing workloads while maintaining acceptable performance. In an e-commerce platform, the workload may grow as the product catalogue, customer activity, and sales history expand.

Estimate the impact of growth

When input size changes from 100,000 to 1,000,000, it increases by a factor of 10. A □(n) algorithm needs about 10 times as much work; a □(n log n) algorithm needs somewhat more than 10 times; and a □(n^2) algorithm needs about 100 times as much work.

These are asymptotic estimates, not exact wall-clock predictions. Actual time also depends on implementation details, hardware, data distribution, and constant factors.

Use the operation budget

If a server performs 10■ basic operations per second, an estimated operation count can be divided by 10■ to obtain an approximate running time in seconds. A ten-minute processing window is 600 seconds, equivalent to about 6×10^{1}■ basic operations under this assumption.

Recommendations for ShopSphere

A pairwise duplicate detector has quadratic growth and should be reconsidered for a very large catalogue. Hash-based grouping by seller and normalized product title can reduce expected work substantially.

A sorted index with binary search is more scalable for lookups than scanning every item, provided the index remains sorted and its maintenance cost is managed.

For recommendation and analytics modules, compare recurrence-based growth against the processing deadline. Remove unnecessary replicated work where possible, profile the implementation, and validate estimates with realistic tests.

Conclusion

Asymptotic analysis provides a systematic way to compare algorithms, predict growth, and choose redesign priorities. The best decision considers time complexity, memory use, practical operation counts, and the platform's required processing window.

: EXACT COUNTING OF OPERATIONS

Counting comparisons in nested loops

When an algorithm compares every item with every other item, the total work depends on the number of distinct pairs. A pair should be counted only once when the purpose is to compare each product with every later product.

For n items, the first item is compared with n □ 1 later items, the second with n □ 2, and so on. The total is (n □ 1) + (n □ 2) + … + 2 + 1.

Using the arithmetic-series formula, the number of comparisons is n(n □ 1)/2. Expanding gives (n² □ n)/2. The highest-order term is n², so the tight time complexity is □(n²).

Why exact counts are useful

An exact count gives more information than a complexity label alone. It can be substituted into a practical input size to estimate the work before running the program.

For large catalogues, however, even a correct quadratic algorithm can be too expensive. The exact formula makes this risk visible and supports a clear redesign decision.

GEOMETRIC SERIES AND DOUBLING LOOPS

Doubling values

A loop that doubles its counter follows the sequence 1, 2, 4, 8, 16, and so on. After k doublings, the counter has value 2■.

The loop stops once the counter reaches or exceeds n. Therefore, the number of doublings is approximately log■ n, which explains why a doubling loop by itself has logarithmic complexity.

Summing the work of an inner loop

If the inner loop runs i times while i doubles, the total work is 1 + 2 + 4 + … + 2■. This is a geometric series.

The sum of the series is 2^(k+1) □ 1. If the last value is just below n, the total is less than about 2n, so the overall complexity is □(n), not □(n log n).

This demonstrates why multiplying the maximum inner-loop cost by the number of outer-loop iterations can be misleading. The inner loop does not perform n operations on every outer iteration.

: BEST, AVERAGE AND WORST CASES

Best case

The best case is the input condition that causes an algorithm to perform the least work for a given input size. In sequential search, the best case occurs when the target is the first item, requiring one comparison.

Worst case

The worst case is the condition that causes the greatest work. In sequential search, this occurs when the target is last or absent, so as many as n comparisons may be required.

Worst-case analysis is useful for service guarantees because it considers a situation in which the algorithm takes its maximum expected amount of work.

Average case

Average-case analysis calculates expected work using assumptions about how likely different inputs are. If a successful sequential search is equally likely to find any position, the average number of comparisons is (n + 1)/2.

If a search succeeds with probability p and succeeds at a uniformly distributed position, its expected comparison count is p(n + 1)/2 + (1 □ p)n. This model assumes an unsuccessful search checks all n entries.

Comparing search methods

Binary search has logarithmic worst-case comparison growth when the data is sorted. Sequential search does not require sorting, but its worst-case comparison count grows linearly with the number of items.

MASTER THEOREM WORKED EXAMPLES

Example 1: T(n) = 4T(n/2) + n

Identify a = 4, b = 2, and f(n) = n. Calculate n^(log_b a) = n^(log■ 4) = n².

The function f(n) = n grows polynomially more slowly than n². The first standard Master Theorem case applies, giving T(n) = □(n²).

Example 2: T(n) = 2T(n/2) + n

Here a = 2, b = 2, and f(n) = n. Calculate n^(log■ 2) = n. The recursive contribution and the additional work have the same order.

The second standard case applies, giving T(n) = □(n log n). This is the familiar growth rate of merge sort when merging at each level takes linear time.

Interpreting the result

The number of recursive subproblems matters as much as the size of each subproblem. Four subproblems of half size create more total recursive work than two subproblems of half size.

In ShopSphere, removing unnecessary replicated partitions can therefore change the growth rate of the recommendation module from quadratic to n log n under the recurrences given in the question.

RECURSION TREE FOR n LOG n WORK

Work at each level

For T(n) = 2T(n/2) + n log n, each node splits into two subproblems. At depth j, there are 2■ subproblems, each with size n/2■.

Each subproblem contributes work proportional to (n/2■) log■(n/2■). Multiplying by the 2■ nodes gives total level work proportional to n(log■ n □ j).

Adding all levels

The tree has approximately log■ n levels. The total non-recursive work is proportional to n times the sum log■ n + (log■ n □ 1) + … + 1.

The sum of the decreasing integers is proportional to (log n)². Multiplying by n gives □(n(log n)²).

This recurrence contains an extra logarithmic factor in f(n), so it is important not to assume that every recurrence with two half-size subproblems has □(n log n) complexity.

PROVING THETA AND REJECTING O(n)

Definition of □

A function f(n) is □(g(n)) if there are positive constants c■, c■ and a threshold n■ such that c■g(n) □ f(n) □ c■g(n) for every n □ n■

.

The lower inequality proves that f(n) does not grow asymptotically slower than g(n). The upper inequality proves that it does not grow asymptotically faster. Together, they establish a tight bound.

Applying the definition

For f(n) = 3n² + 10n log■ n + 50, use the fact that log■ n □ n for integers n □ 1. Thus n log■ n □ n².

For n □ 1, the constant 50 is at most 50n². Therefore f(n) □ 3n² + 10n² + 50n² = 63n². Also, f(n) □ 3n².

Choose c■ = 3, c■ = 63, and n■ = 1. These inequalities show f(n) = □(n²).

Why f(n) is not O(n)

If f(n) were O(n), there would be a constant C such that f(n) □ Cn for all sufficiently large n. But f(n) □ 3n², and 3n²/Cn = 3n/C grows without bound as n increases.

No fixed C can keep 3n² below Cn for all sufficiently large n. Therefore f(n) is not O(n).

OPERATION COUNTS AND TIME ESTIMATES

Converting operations to time

The question assumes that a server executes 10■ basic operations per second. Under this simplified assumption, estimated running time in seconds is the number of basic operations divided by 10■.

For example, 10■ operations correspond to approximately 10 seconds, while 6×10^{1}■ operations correspond to approximately 600 seconds, or 10 minutes.

Comparing the input sizes

The catalogue grows from 100,000 products to 1,000,000 products, a tenfold increase. A linear process is expected to require about ten times the work.

A quadratic process is expected to require about one hundred times the work because $(10n)^2 = 100n^2$. A logarithmic process grows much more slowly, while an n log n process grows somewhat more than linearly.

Limitations of estimates

These calculations are theoretical estimates, not guaranteed real-world timings. Real systems have memory access costs, network delays, database overhead, cache effects, and different costs for different operations.

The estimate is still useful for identifying algorithms that are clearly too expensive and deciding which modules should be tested or redesigned first.

.

: Mathematical Analysis of Searching Algorithms

- Mathematical analysis helps determine the number of operations performed by an algorithm as the input size increases.
- Let \(n\) represent the total number of records stored in the ShopSphere database.
- In linear search, the number of comparisons increases proportionally with the number of records.
- In binary search, the search space is reduced by half after each comparison.
- In hash-based searching, the expected number of operations can remain nearly constant under suitable conditions.
- These mathematical models help estimate algorithm performance before implementing the program.
- The analysis also helps identify which algorithms are suitable for databases containing millions of records.

Derivation of Linear Search Complexity

- Linear search examines the elements sequentially until the target is found or the database ends.
- In the best case, the target is located in the first position, requiring only one comparison.
- In the worst case, the target is located in the last position or does not exist, requiring n comparisons.
- If the target is equally likely to occur at any position, the average number of comparisons for a successful search is:

$$ C_{\text{avg}}=\frac{1+2+\cdots+n}{n} $$

Using the sum of the first n natural numbers:

$$ C_{\text{avg}}=\frac{n+1}{2} $$

- Therefore, the average and worst-case time complexity are $O(n)$.
- This makes linear search less suitable for frequent searches across very large databases.

Derivation of Binary Search Complexity

- Binary search operates on a sorted collection of records.
- Each comparison eliminates approximately half of the remaining search space.
- If the original search space contains n elements, its size after successive steps is approximately:

$$ n,\frac n2,\frac n4,\frac n8,\ldots,1 $$

- After k divisions, the remaining search space is approximately $n/2^k$.
- The search terminates when the remaining space contains at most one element.

$$ \frac{n}{2^k}\leq 1 $$

$$ n\leq 2^k $$

$$ k\geq \log_2 n $$

- Therefore, binary search has logarithmic time complexity, $O(\log n)$.
- Its efficiency depends on maintaining the records in sorted order.

Comparison of Algorithm Growth Rates

- Algorithm growth rates describe how execution requirements change when the dataset becomes larger.
- Constant-time operations do not increase significantly with input size.
- Logarithmic algorithms increase slowly because they reduce the remaining problem size at each step.
- Linear algorithms increase proportionally with the number of records.
- Linearithmic algorithms commonly occur in efficient comparison-based sorting methods.
- Quadratic algorithms can become expensive because their operations may increase with the square of the input size.

For ShopSphere, logarithmic searching or expected constant-time lookup can offer substantial advantages over sequential searching when the database contains one million or more records.

: Recursive Algorithms and Recurrence Relations

- A recursive algorithm solves a problem by calling itself on a smaller version of the same problem.
- Recursive binary search divides the current search interval into two parts and continues with only one part.
- The process terminates when the target is found or the search interval becomes empty.
- A recurrence relation describes the running time of a recursive algorithm in terms of a smaller input.
- For recursive binary search, the recurrence is:

\[T(n)=T(n/2)+c \]

where \(c\) represents the constant work performed outside the recursive call.

- The base case takes constant time.
- Each recursive call reduces the problem size by approximately half.
- Solving the recurrence gives \(T(n)=O(\log n)\).
- Recursive implementations use additional call-stack memory compared with iterative implementations.

: Solving Recurrences Using Substitution

- The substitution method is used to solve recurrence relations by repeatedly expanding the recurrence.
- Consider the recurrence for binary search:

$$ T(n)=T(n/2)+c $$

Expanding repeatedly gives:

$$ T(n)=T(n/4)+2c $$

$$ T(n)=T(n/8)+3c $$

- After k expansions:

$$ T(n)=T(n/2^k)+kc $$

- The base case occurs when $n/2^k=1$.
- Therefore, $k=\log_2 n$.
- Substituting this value gives:

$$ T(n)=T(1)+c\log_2 n $$

- Since $T(1)$ and c are constants, the resulting complexity is $\Theta(\log n)$.
- This method explains why recursive binary search performs efficiently on large sorted datasets.
-

: Master Theorem in Algorithm Analysis

- The Master Theorem provides a method for solving certain divide-and-conquer recurrence relations.
- A common form is:

$$ T(n)=aT(n/b)+f(n) $$

- Here, a represents the number of recursive subproblems, b represents the factor by which the input size is reduced, and $f(n)$ represents the work outside the recursive calls.
- The theorem compares $f(n)$ with $n^{\log_b a}$.
- Depending on their relative growth rates, different complexity results can be obtained.
- For binary search, the recurrence has $a=1$, $b=2$, and constant additional work.
- This produces logarithmic time complexity.
- The standard Master Theorem does not apply to every recurrence, including many recurrences with irregular subproblem sizes or $T(n-1)$ dependencies.

Effect of Database Growth on Performance

- ShopSphere must remain efficient as its records increase from thousands to millions.
- Linear search may require examining a large fraction of the database for each unsuccessful search.
- Binary search requires far fewer comparisons, provided the records are sorted.
- Hash tables and database indexes can support rapid exact-match searches.
- Increased data volume may also increase memory requirements and preprocessing costs.
- Frequent updates can affect the performance of sorting-based and indexed structures.
- Testing at different input sizes helps reveal whether an algorithm scales as expected.
- A scalable design should consider both search speed and the cost of maintaining the data structure.

Performance Evaluation and Result Interpretation

- Experimental evaluation measures how algorithms perform when executed on actual datasets.
- Each algorithm should be tested using the same input sizes and equivalent search targets.
- Execution time provides information about practical speed.
- Comparison counts help explain the amount of work performed independently of some hardware differences.
- Preprocessing time should be recorded separately and included when calculating total cost.
- Repeated trials help reduce the influence of temporary system variations.
- Graphs can show how execution time and comparisons change as input size increases.
- Differences between theoretical and measured results may occur because of processor speed, memory access, compiler optimisation, and system load.
- Experimental evidence should support the final algorithm selection rather than relying only on theoretical complexity.

Slide 40: Final Scalability Recommendations for ShopSphere

- The algorithm must be selected according to the type and frequency of search requests.
- Linear search can be retained for small collections or occasional searches where simplicity is important.
- Binary search is appropriate for sorted data when its preprocessing and update requirements are manageable.
- Hash tables or database indexes should be evaluated for frequent exact-match lookups.
- The design should account for one million existing records and the addition of new records each day.
- The system must be evaluated under the expected workload of 50,000 search requests per hour.
- Performance testing should include average traffic, peak traffic, successful searches, and unsuccessful searches.
- The nightly processing budget should include preprocessing, updates, and other required operations.
- Monitoring and periodic performance testing can identify bottlenecks as the platform grows.

REDESIGN STRATEGIES AND CONCLUSION

Improving duplicate detection

The pairwise method compares every product with every later product. A more scalable strategy is to group products by a combined key such as seller identifier and normalized title.

A hash table can support expected near-linear grouping under suitable assumptions. Normalizing titles and choosing a reliable key are important so that differences in capitalization or spacing do not create missed duplicates.

Improving product search

Sequential search is simple and works with unsorted data, but it may inspect the entire catalogue. Binary search can reduce comparisons on a sorted index to logarithmic growth.

Maintaining a sorted index has a cost when products are inserted or updated, so the platform should compare lookup frequency with index-maintenance requirements.

Improving analytics and recommendations

For recursive modules, use the recurrence to understand how work scales. Avoid unnecessary replication, select suitable data structures, and profile important operations on representative workloads.

A proposed redesign should be evaluated using both asymptotic complexity and actual performance measurements. The goal is to meet the nightly processing deadline while preserving correct results.

Conclusion

Algorithm analysis connects mathematical growth rates to practical engineering decisions. Exact counts, asymptotic bounds, recurrence solving, and time estimates help explain why some designs remain efficient while others become bottlenecks as the catalogue grows.

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Accuracy of Solution	20	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Mark Evaluation:

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20				
Application of Concepts	30				
Problem-Solving Approach	20				
Accuracy of Solution	20				
Clarity	10				
Total Marks					

Code of Conduct

I Chundi.Bhavagna (1971260034) (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Chundi .Bhavagna Date: 12-10-2026